o5s: Verified Patch Admission and Conflict-Preserving Version Control for Agent Swarms

The h5i Team
h5i.dev

ABSTRACT

Coding agents now work on shared repositories in groups of tens or hundreds, unattended, each producing a change every few seconds. Git's integration workflow serializes these changes through a rebase loop: every successful push rejects every other agent's next push, and a textual conflict during rebase stops an agent that has no human to resolve it. Patch-based version control (Darcs, Pijul) removes the loop by making a history a dependency-closed set of immutable patches and by keeping conflicts in the repository as data, but these systems are implemented without a proof that the code admitting and merging patches follows the model, and existing verification work on replicated data proves abstract algorithms rather than executing code.

We present o5s, a patch-based collaboration server for agent swarms whose Rust implementation is formally verified in Lean 4. Our contribution is twofold. First, a bounded formal model of patch-based collaboration: a repository state, its commands, a review-gated admission rule under which a patch joins a channel only with an approval and a passing check report from identities other than its author, and a graph document model whose views retain concurrent edits and their conflicts. Second, a Rust implementation whose decision kernel is translated to Lean 4 and proved to implement that model: the registry transition refines the specification and preserves dependency closure and independent review in every reachable state; materialization yields the same view, conflicts included, for every delivery order of the same patches; and the causal heads a workspace names cover the snapshot it saw. The same kernel runs over a local log, PostgreSQL, and a Cloudflare Worker that keeps Git projections and per-agent workspaces in Cloudflare Artifacts, so agents keep working in ordinary Git checkouts. On one host, 1,000 agents editing adjacent lines of one file integrate at 574 patches per second, where unattended Git integrates 9.8 per second and stops 569 agents at rebase conflicts.

KEYWORDS

version control, patch theory, agent swarms, Lean 4, refinement, admission control

1 INTRODUCTION

Coding agents have changed who edits a repository and how often. A task is no longer one developer on one branch for a day but a swarm of agents, each assigned a subtask, each committing every few seconds, with no human watching the integration step. Hosting providers are rebuilding Git platforms around this client [13].

Git was not designed for this client. Its integration step is a fast-forward of a shared branch. When many agents work against the same branch, each one commits, pushes, and is rejected because another agent's push moved the branch, so it rebases and pushes again. Every successful push rejects everyone else's next attempt, so the throughput of the branch is bounded by the rebase loop whether or not the edits conflict. When they do conflict, the rebase stops with a textual conflict, and an unattended agent has no way to continue.

With 1,000 agents each replacing a different line of one file (Section 8), unattended Git integrates 9.8 changes per second and stops 569 of the agents at rebase conflicts, because its three-way merge treats edits on adjacent lines as overlapping. Spacing the edits four lines apart removes the conflicts but not the loop: all 1,000 integrate, at 5.0 per second, after 55,773 rejected pushes. Hosting services add merge queues and bots on top of this loop, but they do not change its shape.

One alternative is the patch-theory-based approach of Darcs and Pijul, which treat a change as an immutable patch with explicit dependencies, keep a history as a set of patches rather than a sequence of commits, and keep conflicts inside the repository until someone resolves them [4, 17]. Jujutsu stores conflicted states in commits for the same reason [12]. Integration then adds a patch to a set and rewrites nothing, so there is no loop, and a conflict is data that a later patch resolves rather than an error that stops the client. Mimram and Di Giusto and Angiuli et al. give this design a theory: a merge representation must contain unresolved objects [2, 14].

What none of these systems provide is assurance that the code implements the model. The decisions a collaboration server makes, whether a patch is valid, what a document now contains, which patches a channel holds, are trusted by every participant and cannot be checked by them. The properties that make a patch-based history work are universal: materialization must give the same view for *every* order in which the same patches arrive, and an admission rule must hold in *every* reachable state, which tests over a few permutations do not establish.

Version-control infrastructure also has a record of not enforcing its stated rules: Git's July 2025 release fixed submodule-path handling, bundle protocol injection, and credential-helper memory safety, among other issues [3]; a push-option flaw in GitHub allowed server-side command execution, CVE-2026-3854 [18]; GitLab 17.3.3 fixed a SAML authentication bypass, CVE-2024-45409 [8]. Such gaps mattered less when the clients were people, but agents find and use them. In the Hugging Face incident of July 2026, OpenAI's evaluation agents turned a shared package repository into a coordination channel, obtained internet access through it, and compromised third-party systems, with reward hacking and the adoption of peers' goals among the contributing behaviors [16]. A repository service that admits agent-written code should expect the same behavior from its clients, and its admission rule should hold against it.

Verified work on replicated data is the closest prior assurance: Gomes et al. prove strong eventual consistency of CRDTs in Isabelle/HOL [9], but the object of proof is an abstract algorithm

under a network model, convergence is on text rather than on a conflict-bearing view, and admission is not part of the model.

This paper provides that assurance for a bounded model, in two parts.

A formal model of review-gated, patch-based collaboration (Section 4). The model has a repository state (patches, dependencies, approvals, check reports, channels), a small set of commands, and an admission rule: a patch joins a channel exactly when it has an approval and a passing check report from identities other than its author and all its dependencies are already members. Text is a graph of stable atoms with six operations (insert, delete, restore, order, keep-insertion, retract). A document's view is a set of layers and conflicts, and it yields a byte string only when every pair of live atoms is ordered and no delete/insert pair is open. The model fixes which patches a patch may refer to (its causal past), which is what makes validity and views independent of delivery order.

A Rust implementation proved to implement the model (Sections 5 and 6). o5s is built with h5i-app [10], a Rust web framework that separates a service into a *kernel*, the pure function that makes every decision, and a *shell* that handles HTTP, authentication, and storage and never decides anything. Charon and Aeneas [1, 11] translate the kernel to Lean 4 [5], and handwritten, axiom-free proofs establish that the extracted transition refines the model and preserves an invariant containing dependency closure and independent review (Theorem 1), that admission writes a member only with an approval and a passing report present at admission (Theorem 2), that the extracted materializer returns the same decoded view for every delivery order (Theorem 3), and that the extracted head computation returns patches whose causal past is exactly the channel (Theorem 4). The difficulty is in the refinement: the executing document uses dense indices assigned in arrival order and bounded machine integers, while the model is stated over patch identities and unbounded sets, so the proof carries a representation relation across every table and makes the size assumptions explicit. The shell runs the kernel unchanged over a local write-ahead log, PostgreSQL, and a Cloudflare Worker. The Worker keeps Git projections of every channel and a workspace repository for every agent in Cloudflare Artifacts, so agents work in plain Git checkouts and never see the registry.

We evaluate on the repository's recorded single-host comparison with Git (Section 8). Across four workloads with 1,000 agents, o5s admits every proposal, retains same-line edits as explicit alternatives instead of stopping 999 agents, and integrates disjoint edits of one file at 574 patches per second through its API and 145 through its command-line client, against 9.8 for unattended Git. The measurements exercise only the line-edit path, and multi-host, long-history, and adversarial evaluations remain open.

In summary, we make the following contributions:

- A formal model of patch-based collaboration with review-gated admission and a graph document whose views retain conflicts (Section 4).
- Machine-checked proofs that the extracted Rust kernel refines this model, preserves its invariants, enforces the admission gate, materializes independently of delivery order, and computes exact causal heads (Section 5).
- A server that runs the verified kernel over a local log, PostgreSQL, and Cloudflare Workers with Artifacts, and interoperates with ordinary Git workspaces (Section 6).
- Measurements against unattended Git for 1,000 agents on one host, and an explicit account of what remains trusted and unmeasured (Sections 3 and 8).

o5s is open source, and the implementation, the Lean proofs, and the measurement reports are available online [15].

2 BACKGROUND

2.1 Patch-based version control

A Git commit names a tree and its parent commits, and its identity is the hash of all of them [6]. A branch is a pointer to one commit, so integrating two independent commits means making one a descendant of the other: a merge commit, or a rebase that re-creates one commit on top of the other with a new identity [7]. Review, checks, and signatures attach to identities, so whether they survive a rebase is a policy of the hosting service.

A patch-based system keeps the change itself. In Darcs, a patch is a change with the context it needs, and two patches that do not depend on each other commute, so a repository is a set of patches rather than a sequence [4]. Pijul represents a file as a graph of text blocks with stable identities. A patch adds or deletes vertices and edges, names the patches it depends on, and conflicting patches simply coexist in the graph until a later patch resolves them [17]. Pijul calls a named set of patches a *channel*. o5s takes its vocabulary and its three principles from this tradition: a history is a dependency-closed set, integration never rewrites a patch, and a conflict is data.

2.2 Verifying Rust with h5i-app

Aeneas translates a Rust program, via the Charon front end to the compiler's intermediate representation, into a pure Lean 4 function in which borrows are eliminated, integers carry their machine widths, and failure is a result monad [1, 11]. Proofs are then written in Lean about the translated function, so a theorem is about the Rust that runs, up to the correctness of the translation.

h5i-app [10] is a Rust web framework built for this workflow. Its convention is that every decision of a service is one pure function, the *kernel*,

$$\mathit{transition}(a, s, c) \in \mathsf{Ok}(W, \mathit{reply}) \cup \mathsf{Err}(e), \tag{1}$$

taking a *principal* a (the authenticated caller), a *snapshot* s (the rows of one tenant), and a *command* c, and returning either a refusal or a *write set* W of keyed row updates with a reply. The *shell* around the kernel (HTTP routing, token authentication, storage) loads s, calls the kernel, and commits W atomically, so application code never holds a database connection. The row types are declared once with the `schema!` macro, which generates both the Rust storage mapping and the Lean row types.

2.3 A running example

Two agents, A and B, are told to add a line to the same function of one file. The file's current text is two atoms, a and b (Fig. 1, state 1). Each agent inserts its line between a and b: A inserts x, B inserts y.

In Git, both commit against the same base. A pushes first and the branch moves. B's push is rejected, so B rebases, and because

both changes touch the same hunk, the rebase stops with a conflict marker in the file. *B* has no instruction for this case and stops, and the task stays half done until a person looks.

In o5s, both agents propose patches that depend on the same exported snapshot. A reviewer approves both, a check reports pass on both, and both are admitted to the channel. The channel is now the set $\{p_A, p_B\}$ plus their dependencies, and neither patch was rewritten. The document's view (state 2) has an order conflict $\{x, y\}$ and no bytes. A third agent, or a person, proposes a resolution patch *R* that depends on both and adds the single operation **Order**(x, y). Once *R* is admitted, the view has bytes *a x y b* (state 3). Sections 4 and 5 return to this example.

3 ASSUMPTIONS AND THREAT MODEL

The theorems of Section 5 are about four things: the identity of every registered patch, the membership of every channel, the view of every document including its retained conflicts, and the separation of one organization's repositories from another's. They hold for every input that satisfies the verified functions' explicit representation and size assumptions, which includes inputs from an authenticated agent, or several colluding ones, that submit malformed or conflicting proposals, replay requests, approve or check their own proposals, address another organization's repository, or deliver independent patches in any order.

Four components are trusted. *Authentication*: the caller's organization and user identity come from a signed token. *Native preparation*: the SHA-256 identity of a patch body is computed from its canonical serialization, paths are validated, and identities are interned into dense indices outside the kernel. *Storage*: the engine returns the rows a command asks for and commits the returned writes atomically. *Toolchain*: the extraction tools, the Lean kernel, the Rust compiler, and allocation behavior. A database administrator, a stolen signing secret, an incorrect interning table, or a compromised Git implementation can therefore defeat the application-level goal without contradicting any theorem.

Independence means that the approving reviewer and the check reporter are authenticated identities different from the patch author, although they may coincide with each other. The model does not establish that a reviewer is human, that review is honest, or that a reported check was actually run. An agent holding two credentials, or two agents cooperating, can satisfy independence. Availability, sandbox containment, and the semantic correctness of the combined program are outside the guarantees. The Hugging Face incident is the reason this threat model treats agents as adversaries. It is not evidence that a verified VCS would have prevented that intrusion, and we make no such claim.

4 THE MODEL

The model has four parts. Section 4.1 defines the state of a repository: the registered patches, their dependencies, the approvals and check reports on them, and the channels. Section 4.2 gives the commands that change that state and the admission rule, Equation 3, that a merge enforces. Section 4.3 defines the graph document carried inside a patch, as a history of six operations over stable atoms, and the causal past a patch may refer to. Section 4.4 defines the view of a document, that is, its layers, its conflicts, and the condition under which it has a byte string, which is what the materializer computes. Section 4.5 covers the simpler interval representation used by legacy files. Each definition has a counterpart in the Lean development, and Section 5.1 gives the correspondence.

4.1 Patches, dependencies, and channels

Fix a set $\mathcal{U}$ of user identities. A *patch body* is a canonical byte string containing the author's intent, a list of dependencies, and a list of file edits (Sections 4.3 and 4.5). The *identity* of a patch is the SHA-256 hash of its body, so two proposals with the same identity have the same body, and changing any byte of a proposal makes a different patch.

Definition 1 (Repository state). *A repository state is a tuple* $S = (P, D, A, K, C)$ *where*

- P *is a finite set of registered patches, each* $p \in P$ *with an identity* h_p *and an author* $\mathrm{author}(p) \in \mathcal{U}$*, and* $p \mapsto h_p$ *is injective;*
- $D \subseteq P \times P$ *is the dependency relation, where* $(p, q) \in D$ *means that* p *depends on* q *and that* q *was registered before* p*, so* D *is acyclic;*
- $A \subseteq P \times \mathcal{U}$ *is the set of approvals* (p, r)*,* reviewer r approved p*;*
- K *is a partial function from* P *to* $\mathcal{U} \times \{pass, fail\}$*, the latest check report on each patch and who reported it;*
- C *is a finite family of named* channels*, each a sequence* M_c *of patches in admission order.*

Write $\mathrm{deps}(p) = \{q \mid (p, q) \in D\}$ and $q \prec p$ for the transitive closure of D (q *is a strict ancestor of* p). For a set $X \subseteq P$, $\mathrm{Past}(X)$ is X together with all ancestors of its members.

Definition 2 (Dependency closure). *A set* $C \subseteq P$ *is dependency-closed when*

$$p \in C \implies \mathrm{deps}(p) \subseteq C, \tag{2}$$

equivalently $\mathrm{Past}(C) = C$*. Every channel's membership* M_c*, read as a set, is dependency-closed.*

The contrast with Git is the unit of integration. A Git commit's identity covers its parents, so moving a commit onto a new base produces a new identity and review must be re-attached by policy [6, 7]. In *S*, a channel is a *set*, so admitting p to M_c adds p without touching h_p or any earlier member. A fork copies M_c to a new name. Set-based histories remove rebasing of admitted patches, not the need to order concurrent decisions: the implementation still serializes commands on one repository (Section 6.1).

4.2 Commands and the admission rule

The state changes only through the following commands, each issued by an authenticated user u with a role on the repository (*write* ≤ *review* ≤ *admin*, where an unrestricted repository grants *admin* to every member of its organization).

- **Propose**$(h, deps, body)$, role ≥ *write*: if some $p \in P$ already has $h_p = h$, the reply names p and the state is unchanged, so approvals stay attached to the identity they were given for. Otherwise a fresh p with $h_p = h$, $\mathrm{author}(p) = u$, and $\mathrm{deps}(p) = deps \subseteq P$ is registered.
- **Approve**(p), role ≥ *review*: adds (p, u) to A, refused when $u = \mathrm{author}(p)$.

- **Check**(p, result), role $\geq$ *review*: sets $K(p) = (u, \mathit{result})$, replacing any earlier report, refused when $u = \text{author}(p)$.
- **Merge**$(c, [p_1, \ldots, p_n])$, role $\geq$ *review*: admits the listed patches to channel c in order, all or none.
- **Fork**, **Create**, **Grant**: copy a channel, create a repository, set a user's role.

Definition 3 (Admission). *Let M be the membership of c before a merge. Processing p_i with the members admitted so far being $M_i = M \cup \{p_1, \ldots, p_{i-1}\}$, the merge admits p_i exactly when*

$$\exists r.\ (p_i, r) \in A \ \wedge\ K(p_i) = (_, \mathit{pass}) \ \wedge\ \text{deps}(p_i) \subseteq M_i. \quad (3)$$

If any p_i fails Equation 3, the whole command is refused and nothing is written.

By induction on i, the third conjunct of Equation 3 and the closure of M give the closure of $M \cup \{p_1, \ldots, p_n\}$, which is why channels stay dependency-closed. Independence is enforced where A and K are written, so the merge only needs their existence. In the running example, p_A and p_B each satisfy Equation 3 as soon as they are approved and checked, because their only dependencies, the patches of the exported snapshot, are already members, whereas the resolution R depends on both and is admissible only after both are.

4.3 Graph documents: anchors, atoms, and operations

A file edit in a patch body is either an edit of a *graph document*, defined here, or of a legacy *interval document* (Section 4.5). The registry records which representation a path uses the first time it is proposed and refuses the other afterwards.

Let $\mathcal{P}$ be patch identities. A graph document's content is a set of *atoms*: immutable, nonempty byte strings, each identified by the triple (patch that inserted it, ordinal of the inserting operation in that patch, ordinal of the atom in that operation). Authors choose atom boundaries, lines or smaller blocks, and an atom cannot be edited at an offset, only deleted and replaced. An *anchor* is an atom or one of the two sentinels *start* and *stop*.

Definition 4 (History and operations). *A* history *H is a finite set of patches $p = (\mathit{id}_p, \text{deps}_p, \mathit{ops}_p)$ with distinct identities, where ops_p is a list drawn from*

- **Insert**$(l, r, [a_1, \ldots, a_k])$*: new atoms $a_1 \ldots a_k$ as a chain, with order edges $l \to a_1 \to \cdots \to a_k \to r$ between the observed anchors l and r;*
- **Delete**(T)*: a* deletion tag *over the observed atoms T;*
- **Restore**(d)*: cancels the deletion tag written by operation d;*
- **Order**(a, b)*: one order edge $a \to b$ between observed atoms;*
- **KeepInsertion**(d, i)*: acknowledges that insertion i may stay anchored on an atom that deletion d removes;*
- **Retract**(o)*: withdraws the edge written by order operation o.*

Patch p *observes* patch q, written $q \in \text{Obs}(p)$, when $q \in \text{deps}_p$ or q is an ancestor of some dependency. Every anchor, tag, or operation that p refers to must have been written by a patch in $\text{Obs}(p)$. Two patches neither of which observes the other are *concurrent*. Validation of p reads only $\text{Obs}(p)$: it checks that l reaches r along edges written by observed and unretracted operations, and that a new order edge closes no cycle among those edges. Patches that happened to arrive earlier but are not observed by p do not affect whether p is valid, which is what makes validity independent of delivery order. A patch whose dependencies are not yet present is reported as *missing dependencies*, a retryable condition distinct from a validation refusal.

Deletion changes visibility, not identity or anchoring. An atom is *live*, $\text{Live}_H(a)$, when it exists and every deletion tag covering it has been restored. Two concurrent deletions of the same atom are independent tags, so restoring one leaves the other in force. A dead atom still carries its edges, so an insertion that observed the deletion may deliberately anchor on it. A *concurrent* insertion anchored on an atom that an unrestored deletion covers is a *delete/insert conflict* until that deletion is restored or that exact pair (d, i) is acknowledged. An acknowledgement does not cover a later insertion.

4.4 Materialization: layers, conflicts, and clean views

A view of H must order live atoms using paths that may run through dead atoms. Let E_H be the set of edges written by unretracted operations of H plus *start* $\to$ *stop*. The *segment* relation $\text{Seg}_H(u, v)$ holds when there is an E_H-path from u to v whose interior vertices are all dead, and the *live-boundary* relation is

$$\begin{aligned} \text{Sum}_H(u, v) &= \text{Live}_H(u) \wedge \text{Live}_H(v) \wedge \text{Seg}_H(u, v), \\ \text{Prec}_H &= \text{Sum}_H^+, \end{aligned} \quad (4)$$

the transitive closure. A live atom is *blocked* when some Sum_H-cycle reaches it. For unblocked live atoms, $\text{depth}_H(a)$ is the length of the longest Sum_H-chain ending at a, and *layer k* is the set of unblocked live atoms of depth k, which is what a Kahn-style frontier removal computes.

Definition 5 (View). *The view of H consists of its layers, its conflicts, and an optional byte string. The conflicts are: an* order *conflict for every layer with two or more atoms; a* cycle *conflict listing the blocked atoms; and a* delete/insert *conflict for every unacknowledged pair. The byte string is the concatenation of the live atoms in Prec_H order and is defined only when there is no conflict:*

$$\begin{aligned} \text{bytes}_H \ \textit{is defined} \iff\ & \textit{no open delete/insert pair} \\ & \wedge\ \textit{no blocked atom} \\ & \wedge\ \forall u \neq v \ \textit{live}:\ \text{Prec}_H(u, v) \vee \text{Prec}_H(v, u). \end{aligned} \quad (5)$$

The three conditions together say that Prec_H is a strict total order on live atoms. Sorting the atoms of a conflict by identity gives stable diagnostics but is not a tie-breaking text order: no bytes are produced while any conflict is open. A cycle made only of dead atoms blocks nothing, since blocked atoms lie downstream of a cycle through a live atom.

Conflicts are resolved by ordinary patches. Fig. 1 shows the basic case: concurrent inserts x and y between the same anchors both survive, but bytes is undefined until a patch R observing both adds **Order**(x, y). An independent insert z arriving later at the same gap is unordered relative to x and y, and R cannot have ordered what it did not observe, so a new order conflict appears for another resolution. Two concurrent resolutions may add opposite edges

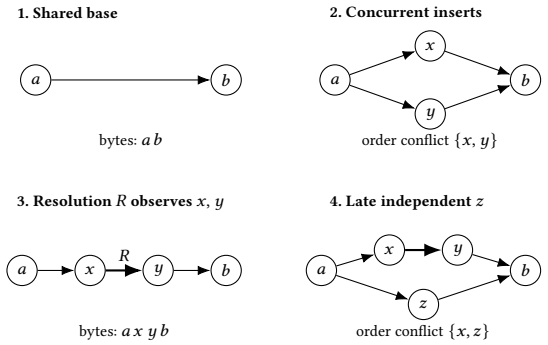


Fig. 1: A resolution orders only what it observed. R adds the edge $x \to y$. An independent z inserted against the original base is unordered relative to x, so the view again has a conflict and no bytes. Transitive edges are omitted.

and create a cycle. **Retract** removes exactly one named edge, and the other order stands.

4.5 Interval documents

The legacy representation edits a file as lines against one immutable base. An edit is a set of intervals (s, e, c), each replacing base lines $[s, e)$ with the lines c. Two nonempty intervals *conflict* when their ranges overlap. A zero-width interval (an insertion at $s = e$) conflicts with any interval whose closed range contains s, so an insertion exactly at the boundary of a replacement is a conflict rather than silently reordered across it. Two intervals with the same range and content are the same edit. A set of in-bounds, pairwise non-conflicting intervals, sorted by start, renders a unique byte string, so concurrent interval edits of one base merge in any order. This model does not reduce graph documents to intervals. It is the representation that the measured workloads use (Section 8) and the one that ordinary Git edits of legacy paths are imported into. Both representations have their own proofs.

5 VERIFICATION WITH H5I-APP

5.1 Instantiating the kernel contract

o5s has two extracted kernels. The *registry* kernel is the h5i-app transition of Equation 1. The *document* kernel, `o5s_kernel`, holds validation, commit, and materialization for graph documents, line merging for interval documents, and the head computation, and is called by the registry kernel and by the shell. Both are written in the Rust subset Charon and Aeneas translate, and both are extracted by h5i-app.

The registry kernel instantiates Equation 1 as follows.

- The principal is $a = (org, user, scope)$: the organization and user from the verified token, and the tenant the shell loaded, which is either one repository or one organization's directory of repository names.
- The snapshot s is one tenant's rows: the repository row with its counters, and the tables `patches`, `dependencies`, `approvals`, `checks`, `channels`, `members`, `paths`, `events`, `bodies`, `listings`, and `roles`. These are the state S of Definition 1 stored relationally: a `patches` row is (id, h_p, author), `dependencies` is D, `approvals` is A, `checks` is K (keyed by patch, so one row per patch), and a `members` row $(c, position, p)$ is one element of M_c.
- The command c is one of the commands of Section 4.2 or a read-only load.
- A write is a keyed upsert of one row (`PutPatch`, `PutApproval`, `PutMember`, …) or a replacement of the repository row, and $\text{commit}(s, W)$ applies them in order.

Alongside the extracted kernel there is a hand-written Lean function $M(a, s, c)$, the *model*, which is Definitions 1–3 written out over the same row types: role checks, hash deduplication, the independence refusals of **Approve** and **Check**, and the fold of Equation 3 for **Merge**. The model is 293 lines of Lean. The document kernel is related in the same way to the model of Sections 4.3 and 4.4.

5.2 Registry theorems

Let $T(a, s, c)$ be the extracted registry transition. The *registry invariant* $\text{Inv}(s)$ is the conjunction of: patch handles and identities are unique and below the repository counters; every dependency names a registered, earlier patch; every approval and check report names a registered patch whose author differs from the reviewer or reporter; every channel's membership is dependency-closed (Equation 2) and every member was approved; and an uncreated repository has no rows.

Theorem 1 (Refinement and invariant preservation). *For every a, s, c: whenever $T(a, s, c)$ returns an outcome, a success $\text{Ok}(W, reply)$ or a refusal $\text{Err}(e)$, that outcome equals $M(a, s, c)$. If $T(a, s, c)$ succeeds and $\text{Inv}(s)$, then $\text{Inv}(\text{commit}(s, W))$. Consequently* Inv *holds in every state reachable from an empty tenant by successful commands; no successful command changes or removes a registered patch; and no successful command removes or reorders a channel member.*

Proof sketch. The extracted helpers (finding a patch by hash, computing the dependency list, the admission fold) are first shown equal to their model counterparts, then `transition` is unfolded by cases on the command. Preservation is one lemma per command. For **Propose**, the fresh handle is the incremented counter, so it cannot collide with an existing row. For **Merge**, induction over the fold shows that each admitted patch's dependencies lie in the members so far, which gives Equation 2 for the result. Reachability is induction over command sequences. The Lean names are `Refine.transition_refines`, `Invariant.inv_preserved`, `reachable_inv`, `patches_immutable`, and `members_append_only`.

Theorem 2 (Admission gate and isolation). *Suppose a* **Merge** *into channel c succeeds in snapshot s with write set W. Then for every member write $(c, _, p) \in W$ the snapshot s contains an approval of p and a check report on p with result pass. In every reachable state those approvals and reports come from identities other than* $\text{author}(p)$. *A repository command succeeds only on a created repository whose organization is a.org, and a directory scope never writes repository rows or vice versa.*

Proof sketch. By Theorem 1 the writes are the model's. The model's fold emits a member write for p only after the two guards of Equation 3 succeed, and those guards read A and K

from s (`Guarantees.merge_gate`). Independence is not part of the merge but of the invariant: the fields `approvals_independent` and `checks_independent` are established by **Approve** and **Check** and carried by `reachable_inv`. The scope statements are `isolation` and `scopes_separate`.

The gate reads the report present *at admission*: because K keeps one report per patch, a later report can replace it, and Theorem 2 does not say that admitted patches have a currently passing report. The report is also about one patch, not about the channel it joins: passing checks on p and on q do not establish anything about the program that contains both. A snapshot-level gate that binds a check to the exact combined channel view is future work.

5.3 Document theorems

The executable document, `document::Doc`, stores the history of Definition 4 with dense indices assigned in arrival order: anchors 0 and 1 are the sentinels, atoms follow in creation order, and tables hold edges with the operation that wrote them, deletion tags, restorations, acknowledgements, and retractions. The relation $\mathrm{Rep}(d, H)$ says that every table of the dense state d is the image of the identity-level history H under the interning map, and $\mathrm{Replayed}(d, H)$ says that d was produced by validating and committing the patches of H through the extracted functions. *Room* conditions bound counts, indices, and output sizes by the machine's integer limits.

Theorem 3 (Order-independent materialization). *Let* $\mathrm{Replayed}(d_1, H_1)$ *and* $\mathrm{Replayed}(d_2, H_2)$ *where* H_1 *and* H_2 *contain the same patches, possibly delivered in different orders, with distinct identities, and both states have room. Then the extracted materializer returns, for* d_1 *and* d_2*, results that decode to the same view of Definition 5: the same layers, the same conflicts, and the same bytes when defined.*

Proof sketch. (1) Simulation: for each of the six operations, the extracted validator refuses exactly when the model does, with the same error, and a validated commit preserves Rep (`DocRefine`). (2) Materialization refinement: the extracted liveness table equals Live_H, the extracted boundary rows are exactly Sum_H, and the Kahn loop's layers and blocked remainder match the depths and blocked set of Section 4.4. Composed, these show that the decoded result satisfies the view specification (`DocMatRefine.materialize_refines`). (3) Model: the view specification depends only on the *set* of patches, and any two views of one history are equal (`view_perm`, `view_unique`). Distinct identities and equal patch sets make H_1 a permutation of H_2, which closes the argument (`extracted_view_converges`).

The model also proves that validation of a patch reads only $\mathrm{Obs}(p)$, that two concurrent admissions commute, and that a patch refused for anything but missing dependencies is refused in every delivery order (`DocModel.validate_local`, `apply_comm`, `admissible_any_order`). Theorem 3 is therefore about every *complete* delivery that respects dependencies, and it does not say that deliveries complete. For interval documents the corresponding results are `Lines.merge_order_independent` and `merged_renders`: two sorted arrangements of the same nonconflicting intervals merge identically and always render.

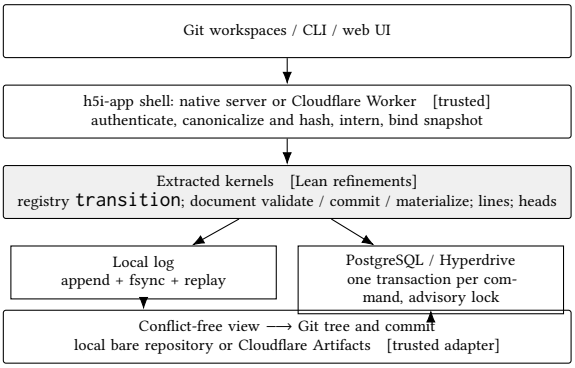


Fig. 2: One decision kernel, two durable backends, and derived Git projections. The shaded layer is the extracted code, and everything else is the shell with explicit obligations.

5.4 Causal heads

A workspace proposal must name, as $\mathrm{deps}(p)$, what its author saw, which is the exported channel view. Naming every member would make dependency lists grow with history. Let $\mathrm{Heads}(C)$ be the members of a dependency-closed C on which no other member directly depends.

Theorem 4 (Heads cover the past). *For a dependency-closed channel* C *whose dependency rows form a DAG, the extracted head computation returns a sublist of* C*, none of whose elements is a dependency of a member, and*

$$\mathrm{Past}(\mathrm{Heads}(C)) = \mathrm{Past}(C) = C. \tag{6}$$

Proof sketch. Every member either is a head or is a dependency of some member. Following dependency edges upward from any member reaches a head because the graph is finite and acyclic, so every member is in the past of some head. Heads are members, so the converse inclusion holds (`Heads.heads_keep_past`). The native workspace adapter is responsible for binding the heads it names to the snapshot it actually exported.

5.5 Trusted computing base

The proofs are complete and axiom-free: no theorem depends on `sorry`, `native_decide`, or any axiom beyond Lean's `propext`, `Classical.choice`, and `Quot.sound`. The Lean they are about is regenerated from the current Rust source and compared byte for byte with the translation the proofs were checked against, so a change to a kernel that is not reflected in the proofs is detected. The full statements, with all hypotheses, are in the repository [15].

6 SYSTEM ARCHITECTURE

6.1 Prepare, decide, commit

A request passes through three stages (Fig. 2). *Prepare*: the shell authenticates the caller, canonicalizes and hashes the proposed patch body, interns the identities it names into handles, and loads the tenant's rows. *Decide*: the registry kernel returns a refusal or a write set, calling the document kernel to validate graph operations and `patch::merge` to admit members. *Commit*: a storage engine

applies the write set atomically. The native server and the Cloudflare Worker call the same `transition`, so the engines differ only in persistence, not in policy.

The local engine appends each decided command, with its SHA-256 checksum, to a write-ahead log, calls `fdatasync` before acknowledging, and replays the log through the kernel's `commit` at startup. The PostgreSQL engine runs each command as one `READ COMMITTED` transaction that first takes a transaction-scoped advisory lock on the tenant (a repository, or an organization's directory). The lock orders all commands of a tenant, and commands of different tenants touch disjoint rows. This replaced an earlier engine that used `SERIALIZABLE` isolation (h5i-app's default), whose predicate locks on shared index pages made unrelated repositories abort each other. Under the lock, a run that previously retried 3,570 of 15,598 attempts retried none.

Two storage optimizations have model-level proofs. A command loads only the rows it can read (`Narrow.narrow_load`: a transition on a partial snapshot equals the transition on the full one when the loaded rows agree on every lookup the command performs), and the PostgreSQL loader is modeled branch by branch and shown to return such rows for reachable states whose dependency lists are stored in key order (`Served.pg_served`). The native server also groups commands that arrive while a tenant's transaction is open into the next transaction, keeping the repository counter row in memory and writing it once. `Group.group_served` and `defer_repo` show that each grouped command sees the state the earlier ones left. Idempotency keys bind a command fingerprint and its reply to the committing transaction, so a retry after an unknown commit outcome returns the stored reply instead of deciding twice.

6.2 Git as projection and workspace

Git is the interface, not the store of record. Agents edit ordinary Git worktrees checked out from an *exported snapshot*: a channel view whose bytes are defined for every path, written as Git trees and a commit whose message records the channel's generation and causal heads. A proposal is the difference between the worktree and that snapshot, with $\mathrm{deps}(p) = \mathrm{Heads}(C)$ by Theorem 4, translated into graph operations against the atoms of the exported view, or into intervals for legacy paths. A view with an open conflict is not exported, so unresolved alternatives never appear as a Git tree. They are resolved by a patch like any other.

The Cloudflare deployment reaches PostgreSQL through Hyperdrive and keeps projections and per-agent workspace repositories in Artifacts. The Worker builds Git objects and pushes a pack, and workspace imports read trees and blobs from the Artifacts binding. A merge records its pending export inside the same database transaction, a scheduled task retries undelivered exports, and generation checks with compare-and-swap reference updates stop an older export from overwriting a newer one. These adapters are tested, not proved.

7 RELATED SYSTEMS

Git. A commit's identity includes its parents [6], so integrating a change onto a moved branch rewrites it [7], and whether review carries over is a hosting policy. o5s admits an unchanged identity into a set. Hosted Git can enforce review and checks through branch protection and hooks. The difference is not the policy but that o5s's policy is an extracted function with a refinement proof and an invariant that holds in every reachable state.

Darcs and Pijul. Darcs reasons about commutation of context-sensitive patches [4]. Pijul represents text as a graph with explicit dependencies and keeps conflicts in the repository [17]. o5s takes stable atom identity, observed context, channels, and conflict retention from this line of work. Its document model is deliberately smaller: six operations, deletion as tags, and order edges with retraction, rather than Pijul's edge-label algebra or Darcs's general patch inversion. The claim is not equivalence to either system but that this bounded model is implemented by proved code.

Theories of patches. Mimram and Di Giusto model merging categorically and show why a merge representation must contain unresolved states [14]. Angiuli et al. represent patch theories as higher inductive types [2]. These works give the design principle that conflicts are objects. o5s's contribution is on the implementation side of that principle: the Rust validator, commit, and materializer are shown to realize the identity-level model of Section 4.

CRDTs and Jujutsu. Sequence CRDTs converge by making concurrent operations commute, and Gomes et al. verified that property in Isabelle/HOL together with a network model [9]. o5s shares the causal-delivery setting but converges on a *conflict-bearing* view rather than on text, and leaves delivery to the shell. Jujutsu also stores conflicted states in commits and allows further work before resolution [12] but does not aim at a verified admission policy.

Table 1: Recorded integrations per second, 1,000 agents on one host. Every o5s run admitted all 1,000 proposals. *API* agents call the HTTP API directly, and *CLI* agents run `o5s propose` in their own Git repository.

Workload	Git integrated	Git/s	o5s API/s	o5s CLI/s
Adjacent lines, one file	431	9.8	574	145
Lines four apart, one file	1,000	5.0	244	48
One new file each	1,000	9.0	1,083	245
Same line, one file	1	—	595*	146*

*Admitted as retained conflict alternatives, so a resolution patch is still needed.

8 PRELIMINARY EVALUATION

Table 1 compares o5s with unattended Git. The host was a 128-thread AMD EPYC 7502 with 251 GiB of memory, running clients and server together. The o5s server used the local-log engine in a release build. Git 2.43.0 used pre-created clones and a `file://` remote, so the Git side paid for no server, authentication, or network. Each run had 1,000 agents with at most 64 requests in flight. A Git agent pushed, and on rejection pulled with rebase and pushed again, and a rebase conflict stopped it. An o5s agent proposed, an independent identity approved and reported a passing check, and patches were merged in batches of 100. The shared-file workloads used interval proposals (Section 4.5), not graph documents, and the check reports were generated by the load driver, not by running a test suite. Final contents were checked in every run.

Where agents edit one shared file, Git's throughput is bounded by the rebase loop regardless of whether the edits conflict (the

four-apart row has no conflicts and 55,773 push retries), and adjacent edits stop 569 of 1,000 agents, whereas o5s admits all of them because disjoint intervals of one base merge as a set. The CLI rows include process start-up and worktree reads for each proposal, whereas the API rows measure the server alone. On the same line, neither system chooses a winner: Git stops 999 agents, each with a local conflict, and o5s keeps all 1,000 alternatives in one place for one reviewed resolution.

The Git baseline is one unattended workflow without a merge queue, custom merge drivers, or human conflict resolution, and the measurements are closed-loop on one host. Multi-host and adversarial evaluations, and load on the graph document path, remain open.

9 CONCLUSION

We presented o5s, a patch-based collaboration server for agent swarms whose Rust decision kernel is formally verified in Lean 4 against a model of review-gated, conflict-preserving version control. The proofs establish that the kernel refines the model and preserves dependency closure and independent review in every reachable state, that a patch is admitted only with an approval and a passing check from identities other than its author, that materialization of a graph document is independent of delivery order, and that the causal heads a workspace names cover the snapshot it saw. The same kernel serves a local log, PostgreSQL, and Cloudflare Workers with Artifacts, and agents interact with it through ordinary Git checkouts. With 1,000 agents on one repository, o5s admits every proposal at 244 to 1,083 integrations per second across the measured workloads, whereas on adjacent lines of one file unattended Git integrates 9.8 changes per second and stops 569 of the agents at rebase conflicts.

REFERENCES

[1] AeneasVerif. 2026. Charon: Interface with the rustc internals. https://github.com/AeneasVerif/charon Accessed October 12, 2026.

[2] Carlo Angiuli, Edward Morehouse, Daniel R. Licata, and Robert Harper. 2014. Homotopical Patch Theory. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP '14)*. ACM, 243–256. https://doi.org/10.1145/2628136.2628158

[3] Taylor Blau. 2025. Git security vulnerabilities announced. The GitHub Blog. https://github.blog/open-source/git/git-security-vulnerabilities-announced-6/ July 8, 2025; updated July 28, 2025.

[4] Darcs project. 2026. Darcs Theory. https://darcs.net/Theory Undated project documentation; accessed October 12, 2026.

[5] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (LNCS, Vol. 12699)*. Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

[6] Git project. 2026. Git Internals: Git Objects. https://git-scm.com/book/en/v2/Git-Internals-Git-Objects Undated Pro Git documentation; accessed October 12, 2026.

[7] Git project. 2026. git-rebase: Reapply commits on top of another base tip. https://git-scm.com/docs/git-rebase Undated online manual; accessed October 12, 2026.

[8] GitLab. 2024. GitLab Critical Patch Release: 17.3.3, 17.2.7, 17.1.8, 17.0.8, 16.11.10. https://docs.gitlab.com/releases/patches/patch-release-gitlab-17-3-3-released/ September 17, 2024.

[9] Victor B. F. Gomes, Martin Kleppmann, Dominic P. Mulligan, and Alastair R. Beresford. 2017. Verifying Strong Eventual Consistency in Distributed Systems. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 109 (2017), 28 pages. https://doi.org/10.1145/3133933

[10] h5i contributors. 2026. h5i: The agent-native web security workspace. https://github.com/h5i-dev/h5i Includes the h5i-app framework (crates h5i-app 0.1.1 and companions); accessed October 12, 2026.

[11] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. *Proceedings of the ACM on Programming Languages* 6, ICFP, Article 116 (2022), 31 pages. https://doi.org/10.1145/3547647

[12] Jujutsu contributors. 2026. First-class conflicts. https://docs.jj-vcs.dev/latest/conflicts/ Undated project documentation; accessed October 12, 2026.

[13] Dina Kozlov, Dillon Mulroy, and Zebulon Piasecki. 2026. We want you to build the next Git platform on Cloudflare. The Cloudflare Blog. https://blog.cloudflare.com/next-git-platform-on-cloudflare/ October 1, 2026.

[14] Samuel Mimram and Cinzia Di Giusto. 2013. A Categorical Theory of Patches. *Electronic Notes in Theoretical Computer Science* 298 (2013), 283–307. https://doi.org/10.1016/j.entcs.2013.09.018

[15] o5s contributors. 2026. o5s: Source, proofs, and recorded measurements at commit d615e77. https://github.com/h5i-dev/o5s/tree/d615e77c60995137ddb47b0bd56544143d76a01c Pinned artifact; full revision d615e77c60995137ddb47b0bd56544143d76a01c.

[16] OpenAI. 2026. The Hugging Face incident and the road ahead. https://openai.com/index/hugging-face-incident-and-the-road-ahead/ August 26, 2026.

[17] Pijul project. 2026. Pijul Manual: Theory. https://pijul.org/manual/theory.html Undated project documentation; accessed October 12, 2026.

[18] Alexis Wales. 2026. Securing the git push pipeline: Responding to a critical remote code execution vulnerability. The GitHub Blog. https://github.blog/security/securing-the-git-push-pipeline-responding-to-a-critical-remote-code-execution-vulnerability/ April 28, 2026; updated April 29, 2026.